Handoff 32: Python unit 3, part 2 (master run S02, EDU-03: running totals, loops inside loops, unit complete)

Date: 2026-11-02 (America/Toronto)
Written by: Claude Code (implementer), in a separate git worktree
For: the human maintainer, and ChatGPT or Claude chat (reviewer) without repository access
Status: **in_review**: one pull request (#37), not merged, not deployed. **No database migration, no new dependency, no API change, no app code change.** Units 1 and 2 are untouched.

> **How to use this file:** upload it with `build.md`, `docs/plan/teaching-design.md`, `docs/plan/lesson-feedback.md`, `docs/plan/browser-checks.md` and handoffs 26 and 27. The pull request diff is the source of truth (`build.md` section 14). The appendix has every card, hint and check case word for word.

Starter prompt for the chat assistant

```
You are reviewing Python Academy stage 32 (EDU-03 part 2): the last two
lessons of Python unit 3 "Repeat and explore" (py.accumulate and
py.nested-loops), which complete the unit. Read the appendix: check that the
wording is calm, original and clear for a grade 9 reader; that the cases catch a
total reset inside a loop, a counter starting from the wrong value, swapped
loops and an inner loop that does not restart, while accepting other correct
programs; that no hint, tagged hint or similar example gives the answer; and
that the cards are accurate Python. Explain at a first-year computer engineering
level.
```

Run plan details

Field	Value
Stage	S02, EDU-03 part 2
Base	`main`. #32 merged before the stage started, so no chaining was needed. `main` was merged in again at `8b5ef45` after #34 merged, and the suites were re-run on that
Part 2 boxes relied on	**None.** B-R2 is not needed, because this part finishes the unit. B-U4 is empty, so handoff 27's questions stay open

1. Step 0

Check	Result
`git fetch --all`, `gh pr list --state all`	#32 and #33 merged by the owner, then #34 during this stage. #35 and #36 open
Lesson files on any branch	`py.accumulate` and `py.nested-loops` were on no branch
Worktree	`../python-academy-u03b`, branch `content/py-u03-part2`. The main folder was not switched

2. Handoff record (build.md template)

Field	Value
Task	EDU-03 part 2: `py.accumulate`, `py.nested-loops`; unit 3 complete
Human goal	A learner can finish unit 3, and is told truthfully that it is finished and that unit 4 is not written yet
Implementer / reviewer	Claude Code / the owner and a chat assistant
Non-goals	A unit project or unit check; construct rule IDs; any change to units 1 or 2 or to earlier unit 3 lessons; `backlog.md` and `decision-log.md`; the root README (the docs sync does these)
Data and compatibility	No migration. New lesson, task and badge IDs only. `/api/catalog` now says `complete: true` for `py.u03`, which is the existing field
Deployment status	Not requested

Files changed

File	Why
`backend/content/py/u03/py.accumulate.json`, `py.nested-loops.json` (new)	The two lessons (appendix)
`backend/content/catalog.json`	`py.u03` has five lessons, no `more_lessons_coming`, and the summary "while and for loops that always finish, running totals, and loops inside loops."
`backend/tests/test_unit3_part2.py` (new, 85 tests), `frontend/tests/unit3-part2.spec.ts` (new, 24 tests), `frontend/tests/fixtures/unit3-part2-api.json` (new, recorded real answers)	Section 6

File	Why
`backend/tests/test_unit2_lessons.py`	The hint audit can now build a loop inside a loop, and put lines below the loops (section 5)
`backend/tests/test_unit3_lessons.py`, `frontend/tests/unit3.spec.ts`	Part 1's tests no longer say the unit is unfinished. "Released so far" is still tested, on a mocked unit still being written
`backend/tests/test_recommend.py`, `test_unit2_part2.py`, `test_content.py`, `test_academy.py`, `frontend/tests/academy.spec.ts`; fixtures `catalog_v1.json`, `lesson-units-api.json`, `unit2-api.json`, `unit2-part2-api.json`, `unit3-api.json`	16 lessons in the course, unit 3 complete
`backend/content/README.md`, `docs/plan/browser-checks.md`	Unit 3 is no longer described as partly released
`docs/run-log.md`, this handoff and its PDF	The record

3. How it works

The story continues with the riddle keeper's game, as handoff 27 question 1 recommended.

Lesson	Challenge	The traps the cases catch
`py.accumulate` "Keep the score"	Ask for the rounds, then each round's points. Print the total, how many rounds scored 0, and the average using `/`	A total or counter reset inside the loop; either one starting from 1; `//` for `/`; prints inside the loop; one round too many or too few; hard-coded answers
`py.nested-loops` "Rooms on the map"	Ask for rows and columns. Print row × column for each room, then the total	Loops the wrong way round; an inner `while` counter that does not restart; a total reset for each row; a stop one early; counting from 0; a missing total; numbers typed into the code

Constructs are not forced. There are no `require` rules. One loop that works out each room's row and column with `//` and `%` passes, and is shown under "Other ways" as correct but less plain.

The output limit. Every expected output is under 200 characters (tested). An inner loop that never moves on reaches the 10,000-character limit and is stopped, also tested in a real browser.

Unit state.

Where	What the learner sees
Catalog	Five lessons, the new summary, still "3 of 12 units open"

Where	What the learner sees
Lesson page, after lesson 5	"Unit complete. You finished every lesson in Repeat and explore. See the course for what comes next."
Home recommendation	"Python Foundations unit 4, Keep track of a world, is next on your path. Its lessons are not written yet."

`RULES_VERSION` stays **2**: only catalog data changed.

4. Decisions and why

Decision	Options considered	Reason	Who decided
Continue the riddle-keeper story	A new story	Recommended in handoff 27, question 1, which is still open	Implementer (**assumption 1**)
No construct rules	`uses-for`, a nested-loop rule	Handoff 27 question 2 recommended none, and any correct program should pass	Implementer (**assumption 2**)
The challenge's rounds are "always at least 1"	Allow 0 rounds	The average would divide by zero, and handling that is not this lesson's idea	Implementer (**assumption 3**)
Average cases all have short exact values, such as 3.4 and 3.75	Include 10 / 3	A long float is a distraction	Implementer
Room coins are row × column, as a number per line	Printing "row col" text	Numbers only, so wording is never judged, and swapped loops still show	Implementer
Part 1's "more lessons coming" test now uses a mock	Delete it	The behaviour still matters for later units released in parts	Implementer
One practice badge per lesson	None per lesson	As in units 1 to 3. Still open as box B-U3	As before

5. Bugs found and how they were fixed

Bug or near-miss	How it was found	Fix
The hint audit could not build a loop inside a loop. Two loop headers were placed one after the other, so a hint spelling out nested loops would not have been caught	Writing the audit for this lesson	`assemble_nested`, which also tries lines below the loops. Every unit's audit now uses it, and earlier results are unchanged
A first draft of the similar example used a list, which unit 4 teaches	Reading it against Part 8	Replaced with doubling gold. A test now rejects `[` and `append` in anything these lessons show
Content validation refused the example's replies without labels	Loading the content	Added "Reply 1" to "Reply 3"
My audit control was badly worded: `... and print(total)` parses as one statement	The control failed	Reworded with "then"
Near-miss: 20 axe `color-contrast` findings on the card label	First scan	A scan made during the card's fade-in. All 20 went once animations finished, and the existing lesson `py.for-range` behaves the same way. Not a real problem, but anyone scanning straight after load will see it
A guessed error title in a browser test	The test failed	The real text is "Division by zero"

I also made one commit with Git hooks switched off, for no good reason. This repository has no hooks, so nothing was skipped, and I redid the commit normally before pushing.

6. How it was tested

Commands and results (each suite alone, macOS, Python 3.12.14, Node 24, after merging `8b5ef45`)

Command	Result
`cd backend && DATABASE_URL=sqlite:///<temp>.db ../../python-academy/backend/.venv/bin/python -m pytest -q`	**785 passed, 11 skipped**
Same with `TEST_POSTGRES_URL=<local PostgreSQL 16 from pgserver>`	**796 passed**
`cd frontend && npx tsc --noEmit && npm run build`	OK
`cd frontend && CI=1 npx playwright test`	**255 passed**

Command	Result
No-mock run, before merging `8b5ef45`: real API (uvicorn), production build, real Pyodide. For each lesson: every card, needs_work, a Python error, an endless loop, an unsupported import and a pass, at 1280 px and 375 px, with axe-core 4.10.3 (WCAG 2.0, 2.1 and 2.2 A and AA, best practices)	Every result as expected. **0 axe violations** once animations finish (section 5); no sideways scrolling

What the tests cover

Required test	Where
Content validation	Browser check v1, 3-second limit, 3 visible and 2 hidden cases, no `require` or `forbid`, three hints, a similar example, two other ways, a new badge; one-comment starters; outputs under 200 characters
What the cards say is true	Each worked example's output, 9 claims on cards and hints, and the predict answers, all run in Python
Correct programs pass	The reference, three creative solutions per lesson and both other ways
Wrong programs fail with the right result	40 samples, including endless, off-by-one, hard-coded and visible-only answers. Every case rule has a sample
Hint audit	Hints, tagged hints and the similar example, alone and combined, with and without the replies being read, again with names swapped for the challenge's. A control proves it catches a hint that spells out the nested loops
Hidden output never sent	Plans and answers, backend and browser
End to end with real Pyodide, each lesson	Every card; a pass with the badge and other ways; needs_work with the tagged hint first; Python errors; an endless loop stopped at 3 s; an unsupported import
XP and badges once	200 XP and 2 badges, nothing for a failed attempt, nothing twice
Unit 1 unchanged	`unit1-unchanged.spec.ts` unchanged and passing
Unit state	"Unit complete."; `complete: true`; the home note; the summary

Mocks: the browser tests answer API calls with recorded **real** answers, and only for an exactly matching request. Python, the worker and the page are real. "More lessons on the way" for a partly released unit is now covered by a modified copy of the catalog, plus `test_recommend.py`.

Deliberate-break controls (each undone; files matched saved copies byte for byte afterwards)

What I broke	Tests that failed
B1. The nested lesson's approach hint spells out the whole walk	**5**: the audit, the renamed audit, the hint-code test and the recordings

What I broke	Tests that failed
B2. The nested check loses the cases that see loop order	**18**, including the wrong samples, which now pass
B3. The accumulate check loses the cases that see a total reset	**11**, including the wrong samples
B4. The check plan sends hidden expected output	**19**: leak tests for every unit, and the recordings
B5. Unit 3 still says more lessons are coming	**12**: catalog, recommendation, recordings and state tests
B6. A recording calls unit 3 not complete	**1** Playwright: the "Unit complete." test

7. Reviewer's questions (plan section 7.2)

1. **Could a learner pass by copying a hint?** The audit says no. It tried every combination of code in the hints, nested and not, with the challenge's own names.
2. **Could a learner fail with a correct program?** Only by printing extra text, as in every unit, and the prompt says "Print just the numbers". These pass: lists, `while` loops, counting scoring rounds instead, `float(total) / rounds`, a formula for the map's total, one loop with `//` and `%`.
3. **Does every failing case name the idea, not the answer?** Yes for hidden cases. Visible cases state their own expected lines, which the learner can already see.
4. **Is every sentence true Python 3.12?** Those that make a claim about output were run. Prose such as "the start line goes above the loop" was read, not run.
5. **One idea per card, readable at grade 9?** Written for it, not measured.
6. **Does the last lesson end the unit truthfully?** Yes, tested. The catalog's project for this unit is not built (S23, blocked), and the ending does not mention one.

8. Known limitations

- **No unit project or unit check** (S23 and S24 need B-A1).
- **Nesting is taught, not enforced.**
- `py.accumulate` **assumes at least 1 round,** as the prompt says. An empty game is not tested.
- **The root README still says part 1 is in review.** The docs sync updates it.
- **Not tested with a real screen reader,** or on Safari or Firefox. Reading level not measured.

9. Live checks for you (after merging and deploying)

You have one test account; your sister can use `/preview` with no account.

- [ ] **The catalog.** `/courses/py`: unit 3 has five lessons and the new summary.
- [ ] **Keep the score, Try it.** Move `total = 0` inside the loop: it prints 8, as the card predicts.

- [ ] **Keep the score, challenge.** Set the total to 0 inside the loop: "Not yet", and the hint says its start line belongs above the loop. Fix it: "Check passed" and the Tally keeper badge.
- [ ] **Rooms on the map, Try it.** Swap the two `for` lines and predict first.
- [ ] **Rooms on the map, challenge.** Swap the loops: "Not yet" for 2 rows and 3 columns. Fix it: "Check passed" and the Map reader badge.
- [ ] **The ending:** "Unit complete."
- [ ] **The home page** after that: unit 4 is next, and its lessons are not written yet.
- [ ] **XP.** 100 XP and one badge per lesson; passing again adds nothing.
- [ ] **Earlier units unchanged.** Someone partway through unit 3 is offered the next lesson.
- [ ] **For your sister:** does "row number times its column number" read clearly? Is the average's `/` clear?
- [ ] **No CSP reports** in DevTools → Console.

10. Questions for you

1. **Assumptions 1 to 3** in section 4. OK? *Recommended, not yet confirmed: keep them.*
2. **Unit 4 next?** It needs B-R2 to release in parts. *Recommended, not yet confirmed: tick B-R2.*
3. **Unit 3's project and check.** Should these wait for B-A1? *Recommended, not yet confirmed: yes, as planned.*
4. **Still open:** handoff 27's questions (B-U4), and B-U3 on one badge per lesson.

11. For the docs sync

Where	What
Backlog EDU-03	in_review for part 2 (#37, handoff 32); part 1 merged, live check pending
Stage table	"32. EDU-03 part 2 | Unit 3 lessons 4 and 5, unit complete | in_review (#37, handoff 32)"
README	Unit 3 released in full, once merged
Decision log	A chapter from sections 4 to 6, and close "Construct rules for nested loops" if question 1 is accepted

12. Next concrete action

Review and merge, then run the live checks.

13. Removing the worktree safely

From the **main** folder, after this PR is merged:

```
cd /Users/DEV/Projects/python-academy
git -C ../python-academy-u03b status --short   # expect nothing, or only ??
frontend/AGENTS.md
rm -f ../python-academy-u03b/frontend/AGENTS.md   # generated by Next.js, not part of the
project
git worktree remove ../python-academy-u03b     # deletes only that folder; refuses if
anything is uncommitted
git worktree prune
git branch -d content/py-u03-part2             # refuses if the branch is not merged
```

Don't delete the folder in the Finder or with `rm -rf`; Git would keep a stale record of it (`git worktree prune` fixes that). If `git branch -d` refuses after a squash merge, check the PR says "Merged" on GitHub, then use `git branch -D content/py-u03-part2`.

Appendix. Every card, hint and check case, word for word

A.1 `py.accumulate`: Keep the score

Concept: Loops · running totals and counters. Description: Add up the points from every round of the riddle keeper's game, count the empty rounds, and find the average. Badge: **Tally keeper** (`py.badge.tally-keeper`, practice).

Card	Text
1. Discover: Build up an answer, one turn at a time	A running total starts at 0 before the loop. Inside the loop, total = total + coins adds this turn's number to what is already there. When the loop ends, total holds the sum of them all. A counter works the same way, but adds 1 each time something happens, often inside an if. The start line goes above the loop, so it runs once: inside the loop it would wipe the total every turn. For an average, divide the total by how many numbers there were, after the loop.
2. Worked example	`# Three chests: add up the coins, then find the average` / `chests = 3` / `total = 0` / `for chest in range(chests):` / `coins = int(input("Coins in this chest? "))` / `total = total + coins` / `print(total)` / `print(total / chests)` Replies: Reply 1: '4', Reply 2: '0', Reply 3: '8'. Output: `12` / `4.0` total starts at 0 and grows: 4, then 4 again, then 12. The two print lines are below the loop, not indented, so they run once, after the last chest. Dividing with / always gives a float, so the average shows as 4.0.
3. Predict	Suppose total = 0 moves inside the loop, as the first line of its block. With the same replies, what would print(total) show? Options: **8** (answer); 12; 0 Why: total goes back to 0 at the start of every turn, so each chest forgets the ones before it. Only the last chest's 8 is left when the loop ends.

Card	Text
4. Try it	Predict, then run: move total = 0 inside the loop, as the first line of its block, and check your prediction. Put it back above the loop. Then add a counter for empty chests: start it above the loop, add 1 inside an if, and print it at the end.
5. Challenge	The riddle keeper's game now has rounds. Ask how many rounds were played (always at least 1), then ask for the points from each round. Print three lines: the total points, how many rounds scored 0, and the average points per round, using /. Print just the numbers. Starter: `# Ask for the rounds, then the points from each round`

Hint	Text
A nudge	You need two things that build up as the loop runs, and both start at 0 above the loop: one for the points, and one for the rounds that scored 0.
A bigger hint	A for loop over range() runs once per round. Ask for that round's points inside it. The three print lines go after the loop, with no indent.
The approach	A pattern with other names: start with distance = 0, and inside the loop use distance = distance + miles. Your loop also needs a counter, and an if to decide when it goes up.
About `TypeError`	A reply from input() is text. Turn it into a number with int(...) before you add it up or use it in range().
About `NameError`	Give each name a value before the loop uses it, and spell it the same way every time.
About `IndentationError`	The lines that repeat start with four spaces. A line inside an if inside the loop starts with eight.
About `EOFError`	Your loop asked for more points than there were rounds. range(rounds) already runs exactly once per round.
About `ZeroDivisionError`	The average divides by the number of rounds played, and that is always at least 1. Check which number comes after the /.
About `py.accumulate.case.one-round`	With one round of 10 points, the lines are 10, 0 and 10.0. A total or a counter that starts above 0 is already wrong here, and // gives 10 where / gives 10.0.
About `py.accumulate.case.three-rounds`	With rounds of 10, 0 and 5, the lines are 15, 1 and 5.0. If your total says 5, it goes back to 0 on every turn: its start line belongs above the loop.
About `py.accumulate.case.no-points`	With two rounds of 0, the lines are 0, 2 and 0.0. Every round that scores 0 counts, even when nothing has been scored yet.

Hint	Text
About `py.accumulate.case.many-rounds`	Check a longer game: use the number of rounds in range() and in the average, and / rather than //, which throws away the part after the point.
About `py.accumulate.case.more-empty`	Check a game with more than one empty round: the counter goes up every time a round scores 0.
Similar example	`gold = 1` / `for day in range(3):` / `gold = gold * 2` / `print(gold)` Another answer that builds up: the gold doubles each day, giving 8 after three days. It starts at 1, because doubling 0 would stay 0. Your challenge adds up every round and counts the empty ones instead.
Other way, after passing	`rounds = int(input("Rounds? "))` / `total = 0` / `empty = 0` / `count = 0` / `while count < rounds:` / `points = int(input("Points? "))` / `total = total + points` / `if points == 0:` / `empty = empty + 1` / `count = count + 1` / `print(total)` / `print(empty)` / `print(total / rounds)` The same totals with a while loop, which needs its own count of the rounds so far. for and range() do that counting for you.
Other way, after passing	`rounds = int(input("Rounds? "))` / `total = 0` / `scored = 0` / `for game in range(rounds):` / `points = int(input("Points? "))` / `total += points` / `if points > 0:` / `scored += 1` / `print(total)` / `print(rounds - scored)` / `print(total / rounds)` Count the rounds that scored instead, and take them away from all the rounds. += is a short way to write total = total + points.

Case	Visibility	Description (shown to learners)	Replies	Expected output
`one-round`	visible	a single round of 10 points	'1', '10'	`10` / `0` / `10.0`
`three-rounds`	visible	rounds of 10, 0 and 5 points	'3', '10', '0', '5'	`15` / `1` / `5.0`
`no-points`	visible	two rounds of 0 points	'2', '0', '0'	`0` / `2` / `0.0`
`many-rounds`	hidden	a longer game	'5', '1', '5', '10', '0', '1'	`17` / `1` / `3.4`
`more-empty`	hidden	a game with several empty rounds	'4', '0', '10', '5', '0'	`15` / `2` / `3.75`

A.2 `py.nested-loops`: Rooms on the map

Concept: Loops · a loop inside a loop. Description: Walk the riddle keeper's map row by row, count the coins in each room, and add them all up. Badge: **Map reader** (`py.badge.map-reader`, practice).

Card	Text
1. Discover: Put a loop inside a loop	A loop's block can hold another loop. The inner loop runs all the way through, every time the outer loop goes round once. So an outer loop of 2 turns and an inner loop of 3 turns runs the inner block 6 times. The outer loop's line starts at the left, the inner loop's line has four spaces, and the inner block has eight. for and range() start the inner loop again by themselves. With while, the inner counter has to be set back to its start inside the outer loop, or the inner loop only runs the first time.
2. Worked example	`# Number the seats: two rows of three` / `for row in range(1, 3):` / `for seat in range(1, 4):` / `print(row * 10 + seat)` Output: `11` / `12` / `13` / `21` / `22` / `23` row starts at 1, and seat runs through 1, 2 and 3. Then row becomes 2, and seat starts again from 1. Multiplying by 10 is just a way to show both numbers at once: 23 is row 2, seat 3.
3. Predict	If the two for lines swapped places, so that seat is the outer loop and row the inner one, what would the first three lines print? Options: **11, 21 and 12** (answer); 11, 12 and 13; 11, 21 and 31 Why: The inner loop finishes first. seat stays at 1 while row runs through 1 and 2, giving 11 and 21. Then seat becomes 2 and row starts again, giving 12.
4. Try it	Predict, then run: swap the two for lines, with row and seat keeping their ranges. Put them back. Then change range(1, 3) to range(1, 4) and predict how many lines print before you run.
5. Challenge	The riddle keeper's map is a grid of rooms. Rows and columns are counted from 1, and a room holds its row number times its column number in coins, so row 2, column 3 holds 6. Ask for the number of rows, then the number of columns. Going through the rows in order, and left to right along each row, print the coins in every room, one per line. On the last line, print the total coins. Print just the numbers. Starter: `# Ask for the rows and the columns, then walk the map`

Hint	Text
A nudge	One for loop goes through the rows. Inside its block, another for loop goes through the columns of that row.
A bigger hint	Rows and columns count from 1, and range() stops before its second number. The total starts at 0 above both loops, and prints below both of them.
The approach	A pattern with other names: for shelf in range(1, shelves + 1): holds for book in range(1, books + 1): one step further in. The rooms need your own names, your own sum, and a total.
About `TypeError`	range() needs whole numbers. Turn each reply into a number with int(...) before you use it.
About `NameError`	Use one name for the rows and one for the columns, and give each a value with input() before the loops.

Hint	Text
About `IndentationError`	Each loop's block goes four spaces further in than its for line: four for the inner for line, and eight for the lines inside it.
About `EOFError`	This program needs only two replies, the rows and the columns. Ask for them once, above the loops.
About `py.nested-loops.case.wide`	With 2 rows of 3 columns, the coins go 1, 2, 3, then 2, 4, 6, and the total is 18. If you see 1, 2, 2, 4, 3, 6, the two loops are the wrong way round. If only the first row printed, the inner counter did not start again.
About `py.nested-loops.case.tall`	With 3 rows of 2 columns, the coins go 1, 2, then 2, 4, then 3, 6. Use the replies in your range() calls, not numbers typed into the code.
About `py.nested-loops.case.single`	A map of one room holds 1 coin, so the lines are 1 and then the total, 1. The total line is always printed.
About `py.nested-loops.case.more-rows`	Check a map with more rows than the examples: the outer range() has to follow the rows reply.
About `py.nested-loops.case.more-columns`	Check a map with more columns than the examples: the inner range() has to follow the columns reply.
Similar example	`for hour in range(1, 3):` / `for minute in range(0, 60, 30):` / `print(hour * 100 + minute)` A clock that strikes every half hour shows 100, 130, 200 and 230: for each hour, the minutes start again from 0. Your challenge goes through rows and columns, counts from 1, and adds everything up.
Other way, after passing	`rows = int(input("Rows? "))` / `columns = int(input("Columns? "))` / `total = 0` / `row = 1` / `while row <= rows:` / `column = 1` / `while column <= columns:` / `print(row * column)` / `total = total + row * column` / `column = column + 1` / `row = row + 1` / `print(total)` The same walk with while loops. Look at column = 1: it sits inside the outer loop, so each row starts again from the first column. for and range() do that for you.
Other way, after passing	`rows = int(input("Rows? "))` / `columns = int(input("Columns? "))` / `total = 0` / `for room in range(rows * columns):` / `row = room // columns + 1` / `column = room % columns + 1` / `print(row * column)` / `total = total + row * column` / `print(total)` One loop over every room, working out its row and column with // and %. It is correct and clever, but two loops say what you mean more plainly.

Case	Visibility	Description (shown to learners)	Replies	Expected output
`wide`	visible	a map of 2 rows and 3 columns	'2', '3'	`1` / `2` / `3` / `2` / `4` / `6` / `18`

Case	Visibility	Description (shown to learners)	Replies	Expected output
`tall`	visible	a map of 3 rows and 2 columns	'3', '2'	`1` / `2` / `2` / `4` / `3` / `6` / `18`
`single`	visible	a map of one room	'1', '1'	`1` / `1`
`more-rows`	hidden	a map with more rows	'4', '3'	`1` / `2` / `3` / `2` / `4` / `6` / `3` / `6` / `9` / `4` / `8` / `12` / `60`
`more-columns`	hidden	a map with more columns	'2', '6'	`1` / `2` / `3` / `4` / `5` / `6` / `2` / `4` / `6` / `8` / `10` / `12` / `63`